Q1) Run a SELECT query on a large 'orders' table (at least 10,000 rows) to find all orders for a specific user_id and measure the query execution time.

```
CREATE INDEX idx_user_id ON
orders(user_id);
```

Q2) Create an index on the user_id column of the 'orders' table and re-run the same SELECT query to measure the new execution time.

Hint: Use CREATE INDEX idx_user_id ON orders(user_id); and compare the times before and after.

```
CREATE INDEX idx_user_id
ON orders(user_id);

SELECT *
FROM orders
WHERE user_id = 101;
```

Q3) Use the EXPLAIN PLAN command to analyze how your SELECT query runs before and after adding the index, and write down the key differences you observe in the output.

```
CREATE INDEX idx_user_id
ON orders(user_id);

EXPLAIN
SELECT *
FROM orders
WHERE user_id = 101;
```

Q4) Write a query for a 'products' table that avoids a full table scan by using an index on the 'category' column to fetch all products in a specific category.

```sql
CREATE INDEX idx_category
ON products(category);

SELECT *
FROM products
WHERE category =
'Electronics';

EXPLAIN
SELECT *
FROM products
WHERE category =
'Electronics';
```

Q5) Suppose your SELECT query on the 'orders' table is still slow even after adding an index. Use EXPLAIN PLAN and research at least one more optimization technique (other than indexing) using an AI tool like ChatGPT or Copilot, and describe how you would apply it.

```sql
EXPLAIN
SELECT order_id,
order_date, amount
FROM orders
WHERE user_id = 101;

EXPLAIN ANALYZE
SELECT order_id,
order_date, amount
FROM orders
WHERE user_id = 101;
```